BSIT 375: Knowledge Discovery and Data Science

Week 3 Lecture Notes: Preparing to Model the Data

Based on Larose & Larose: *Discovering Knowledge in Data (2nd Ed)*

Instructor: Tek Raj Pant

March 2026

Introduction

These lecture notes cover two fundamental algorithms in data science: **K-Nearest Neighbors (KNN)** for classification and regression, and **K-Means clustering** for unsupervised learning. Both methods rely on the concept of proximity (distance) between data points, but they serve different purposes. We will explore their inner workings through step-by-step numerical examples.

1 K-Nearest Neighbors (KNN) Algorithm

Overview

KNN is a simple, instance-based learning algorithm used for both classification and regression. It makes predictions by finding the K training examples closest to a new data point and aggregating their outputs (majority vote for classification, average for regression). It is non-parametric and lazy (no explicit training phase).

Key Concepts

- **Distance Metric**: Typically Euclidean, Manhattan, or Minkowski. Euclidean distance between points $\mathbf{x} = (x_1, \ldots, x_p)$ and $\mathbf{y} = (y_1, \ldots, y_p)$ is:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2}.$$

- **Choice of K**: A small K can be noisy; a large K smoothes boundaries. K is often chosen by cross-validation.

- **Feature Scaling**: Because distances are sensitive to scales, normalization (e.g., min-max or z-score) is crucial.

Algorithm Steps

1. Choose K and a distance metric.
2. For a new point q, compute distances to all training points.
3. Select the K nearest neighbors.
4. For classification: assign the majority class among them. For regression: assign the average of their target values.

Numerical Examples

Example 1: Classification (Predicting Flower Species): Suppose we have a small dataset of flowers with two features (sepal length, sepal width) and three species: Setosa (S), Versicolor (V), and Virginica (Vg). We want to classify a new flower with sepal length = 5.2 cm and sepal width = 3.4 cm. Use $K = 3$ and Euclidean distance.

Table 1: Training Data

Sepal Length (x_1)	Sepal Width (x_2)	Species
5.1	3.5	S
4.9	3.0	S
6.2	3.4	V
5.9	3.2	V
6.5	3.0	Vg
7.0	3.2	Vg

Step 1: Compute distances from new point $q = (5.2, 3.4)$.

$$
\begin{aligned}
d_1 &= \sqrt{(5.2-5.1)^2 + (3.4-3.5)^2} = \sqrt{0.01+0.01} = \sqrt{0.02} \approx 0.1414 \\
d_2 &= \sqrt{(5.2-4.9)^2 + (3.4-3.0)^2} = \sqrt{0.09+0.16} = \sqrt{0.25} = 0.5 \\
d_3 &= \sqrt{(5.2-6.2)^2 + (3.4-3.4)^2} = \sqrt{1.0+0} = 1.0 \\
d_4 &= \sqrt{(5.2-5.9)^2 + (3.4-3.2)^2} = \sqrt{0.49+0.04} = \sqrt{0.53} \approx 0.7280 \\
d_5 &= \sqrt{(5.2-6.5)^2 + (3.4-3.0)^2} = \sqrt{1.69+0.16} = \sqrt{1.85} \approx 1.3601 \\
d_6 &= \sqrt{(5.2-7.0)^2 + (3.4-3.2)^2} = \sqrt{3.24+0.04} = \sqrt{3.28} \approx 1.8111
\end{aligned}
$$

Step 2: Sort distances and pick $K = 3$ smallest:

- Neighbor 1: $d_1 = 0.1414$ (S)

- Neighbor 2: $d_2 = 0.5$ (S)
- Neighbor 3: $d_4 = 0.7280$ (V)

Step 3: Majority vote: two S, one V. Therefore, the new flower is classified as **Setosa**.

Example 2: Regression (Predicting House Price): We have a small dataset of houses with size (in sq.ft) and number of bedrooms, and price (in \$1000). Predict the price of a house with size = 1650 sq.ft and 3 bedrooms using $K = 3$ and Euclidean distance. (Data scaled to [0,1] for fair comparison.)

Table 2: Training Data (Raw and Normalized)

House	Size (sq.ft)	Bedrooms	Price (\$k)	Size (norm)	Bedrooms (norm)
1	1200	2	250	0.00	0.00
2	1400	3	300	0.33	0.50
3	1600	4	350	0.67	1.00
4	1800	3	400	1.00	0.50

Normalization: Size scaled with min=1200, max=1800; Bedrooms scaled with min=2, max=4.

New point: size=1650, bedrooms=3. Normalized: $(1650 - 1200)/(1800 - 1200) = 0.75$, $(3 - 2)/(4 - 2) = 0.5$. So $q = (0.75, 0.5)$.

Compute Euclidean distances:

$$d_1 = \sqrt{(0.75 - 0)^2 + (0.5 - 0)^2} = \sqrt{0.5625 + 0.25} = \sqrt{0.8125} \approx 0.9014$$
$$d_2 = \sqrt{(0.75 - 0.33)^2 + (0.5 - 0.5)^2} = \sqrt{0.1764 + 0} = 0.42$$
$$d_3 = \sqrt{(0.75 - 0.67)^2 + (0.5 - 1)^2} = \sqrt{0.0064 + 0.25} = \sqrt{0.2564} \approx 0.5064$$
$$d_4 = \sqrt{(0.75 - 1)^2 + (0.5 - 0.5)^2} = \sqrt{0.0625 + 0} = 0.25$$

Sorted: $d_4 = 0.25$ (House 4, price 400), $d_2 = 0.42$ (House 2, price 300), $d_3 = 0.5064$ (House 3, price 350). $K = 3$ neighbors: Houses 4,2,3.

Predicted price = average of their prices = $(400 + 300 + 350)/3 = 350$ (in \$1000). So predicted price is **\$350,000**.

2 K-Means Clustering Algorithm

Overview

K-Means is a centroid-based clustering algorithm that partitions n data points into K clusters, each represented by the mean of its points. It minimizes the within-cluster sum of squares (WCSS). The algorithm iteratively assigns points to the nearest centroid and updates centroids until convergence.

Algorithm Steps

1. Choose K (number of clusters).
2. Initialize K centroids randomly (or using heuristics like k-means++).
3. Repeat until centroids do not change:
   (a) Assign each point to the cluster with the nearest centroid (using Euclidean distance).
   (b) Recompute each centroid as the mean of points assigned to that cluster.
4. Output the final clusters.

Choosing the Number of Clusters (K)

A common method is the **elbow method**: run K-means for a range of K, compute the WCSS (sum of squared distances from each point to its cluster centroid), and choose K where the decrease in WCSS begins to flatten.

Numerical Examples

Example 1: Simple 2D Clustering (4 points, K=2): Consider four points: A(1,2), B(2,1), C(4,3), D(5,4). We want $K = 2$ clusters.

Initial centroids (choose first two points): $C_1 = (1, 2)$, $C_2 = (2, 1)$.

Iteration 1:

- Distances to C_1: A:0, B:$\sqrt{(2-1)^2 + (1-2)^2} = \sqrt{2} \approx 1.41$, C:$\sqrt{(4-1)^2 + (3-2)^2} = \sqrt{9+1} = 3.16$, D:$\sqrt{(5-1)^2 + (4-2)^2} = \sqrt{16+4} = 4.47$.
- Distances to C_2: A:$\sqrt{(1-2)^2 + (2-1)^2} = 1.41$, B:0, C:$\sqrt{(4-2)^2 + (3-1)^2} = \sqrt{4+4} = 2.83$, D:$\sqrt{(5-2)^2 + (4-1)^2} = \sqrt{9+9} = 4.24$.

Assign each point to the closer centroid:

- A: closer to C_1 (1.41 vs 0? Actually d(A,C1)=0, d(A,C2)=1.41 → cluster 1)
- B: closer to C_2 (1.41 vs 0 → cluster 2)
- C: closer to C_1 (3.16 vs 2.83? Actually 3.16 ¿ 2.83 → cluster 2) – careful: 3.16 vs 2.83, so cluster 2 is closer.
- D: closer to C_2 (4.47 vs 4.24 → cluster 2)

So clusters: C_1: {A}, C_2: {B, C, D}.

Update centroids:

$$C_1 = (1, 2) \quad \text{(only A)}; \quad C_2 = \left(\frac{2+4+5}{3}, \frac{1+3+4}{3}\right) = \left(\frac{11}{3}, \frac{8}{3}\right) \approx (3.67, 2.67).$$

Iteration 2: Recompute distances to new centroids.

- Distances to $C_1 = (1, 2)$: A:0, B:$\sqrt{2} \approx 1.41$, C:3.16, D:4.47.
- Distances to $C_2 = (3.67, 2.67)$: A:$\sqrt{(1-3.67)^2 + (2-2.67)^2} = \sqrt{7.13 + 0.45} = 2.75$, B:$\sqrt{(2-3.67)^2 + (1-2.67)^2} = \sqrt{2.79 + 2.79} = 2.36$, C:$\sqrt{(4-3.67)^2 + (3-2.67)^2} = \sqrt{0.11 + 0.11} = 0.47$, D:$\sqrt{(5-3.67)^2 + (4-2.67)^2} = \sqrt{1.77 + 1.77} = 1.88$.

Assignments:

- A: d(C1)=0, d(C2)=2.75 → cluster 1.
- B: d(C1)=1.41, d(C2)=2.36 → cluster 1 (since 1.41 ¡ 2.36).
- C: d(C1)=3.16, d(C2)=0.47 → cluster 2.
- D: d(C1)=4.47, d(C2)=1.88 → cluster 2.

Clusters: C_1: {A,B}, C_2: {C,D}.

Update centroids:

$$C_1 = \left(\frac{1+2}{2}, \frac{2+1}{2}\right) = (1.5, 1.5), \quad C_2 = \left(\frac{4+5}{2}, \frac{3+4}{2}\right) = (4.5, 3.5).$$

Iteration 3: Distances to C_1 and C_2:

- A: d(C1)=$\sqrt{(1-1.5)^2 + (2-1.5)^2} = 0.71$, d(C2)=$\sqrt{(1-4.5)^2 + (2-3.5)^2} = 3.81$ → cluster1.
- B: d(C1)=0.71, d(C2)=$\sqrt{(2-4.5)^2 + (1-3.5)^2} = 3.54$ → cluster1.
- C: d(C1)=$\sqrt{(4-1.5)^2 + (3-1.5)^2} = 3.54$, d(C2)=0.71 → cluster2.
- D: d(C1)=$\sqrt{(5-1.5)^2 + (4-1.5)^2} = 4.95$, d(C2)=0.71 → cluster2.

Assignments unchanged. Centroids remain $(1.5, 1.5)$ and $(4.5, 3.5)$. Converged.

Final clusters: **Cluster 1**: A, B; **Cluster 2**: C, D.

Example 2: Clustering with 6 points, K=3: Points: (1,1), (1,2), (2,1), (5,5), (5,6), (6,5). Choose $K = 3$ and initial centroids randomly (for illustration, pick (1,1), (2,1), (5,5)). We'll show the first iteration.

Data and initial centroids:

$$\text{Points: } P_1(1,1), P_2(1,2), P_3(2,1), P_4(5,5), P_5(5,6), P_6(6,5)$$

$$C_1 = (1,1),\ C_2 = (2,1),\ C_3 = (5,5)$$

Compute distances (Euclidean) and assign:

Point	$d(C_1)$	$d(C_2)$	$d(C_3)$	C
$P_1(1,1)$	0	1	$\sqrt{(1-5)^2+(1-5)^2} = 5.66$	
$P_2(1,2)$	1	$\sqrt{(1-2)^2+(2-1)^2} = 1.41$	$\sqrt{(1-5)^2+(2-5)^2} = 5$	
$P_3(2,1)$	1	0	$\sqrt{(2-5)^2+(1-5)^2} = 5$	
$P_4(5,5)$	5.66	$\sqrt{(5-2)^2+(5-1)^2} = 5$	0	
$P_5(5,6)$	$\sqrt{(5-1)^2+(6-1)^2} = 6.4$	$\sqrt{(5-2)^2+(6-1)^2} = 5.83$	1	
$P_6(6,5)$	$\sqrt{(6-1)^2+(5-1)^2} = 6.4$	$\sqrt{(6-2)^2+(5-1)^2} = 5.66$	1	

Clusters: C_1: {P1,P2}, C_2: {P3}, C_3: {P4,P5,P6}.

Update centroids:

$$C_1 = \left(\frac{1+1}{2}, \frac{1+2}{2}\right) = (1, 1.5), \quad C_2 = (2,1), \quad C_3 = \left(\frac{5+5+6}{3}, \frac{5+6+5}{3}\right) = (5.33, 5.33).$$

Further iterations would continue until stability. The final clusters would separate the two natural groups: left bottom points (P1,P2,P3) and right top points (P4,P5,P6) with possibly a refined split. This example shows how centroids move and points are reassigned.

3 Model Performance Evaluation

After building a model, we need to assess how well it performs. The evaluation metrics differ between supervised (KNN) and unsupervised (K-Means) learning.

Evaluating KNN Models

Since KNN is a supervised method, we split the data into training and test sets (or use cross-validation) to obtain unbiased performance estimates.

Classification Metrics: For a classification task, the most common metric is **accuracy**:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}.$$

A more detailed view is given by the **confusion matrix**, which shows true positives, false positives, etc. For multi-class problems, accuracy alone can be misleading when classes are imbalanced; metrics like precision, recall, and F1-score are then used.

Choosing K in classification: One typically performs k-fold cross-validation (e.g., 10-fold) to find the K value that maximizes the average accuracy on the validation folds.

Regression Metrics: For regression, common evaluation metrics are:

- **Mean Squared Error (MSE)**: $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$.
- **Root Mean Squared Error (RMSE)**: $\sqrt{\text{MSE}}$.
- **Coefficient of Determination (R^2)**:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2},$$

  where $\bar{y}$ is the mean of the true values. R^2 ranges from $-\infty$ to 1; a value close to 1 indicates a good fit.

Again, cross-validation is used to select the K that minimizes MSE or maximizes R^2.

Evaluating K-Means Clustering

Because K-Means is unsupervised (no true labels), we use internal validation measures that assess cluster quality based on the data itself.

Within-Cluster Sum of Squares (WCSS): WCSS measures the compactness of the clusters:

$$\text{WCSS} = \sum_{j=1}^{K} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2,$$

where $\boldsymbol{\mu}_j$ is the centroid of cluster C_j. Lower WCSS means tighter clusters. When plotting WCSS against K, the **elbow point** (where the decrease slows) is often chosen as the optimal K.

Silhouette Score: The silhouette score combines both cohesion (how close points are to their own cluster) and separation (how far they are from the nearest other cluster). For a single point i, the silhouette coefficient $s(i)$ is:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where $a(i)$ is the mean distance to other points in its own cluster, and $b(i)$ is the mean distance to points in the nearest neighboring cluster. The overall silhouette score is the

average of $s(i)$ over all points. It ranges from -1 (poor clustering) to 1 (well-separated, compact clusters). A value above 0.5 is generally considered good. One can compute the silhouette score for different K and choose the K that maximizes it.

Both WCSS and silhouette score help in selecting an appropriate number of clusters without any ground truth.

4 Comparison and Summary

- **KNN**: Supervised, lazy, uses all training data at prediction time, sensitive to scale and K, works for both classification and regression.
- **K-Means**: Unsupervised, iterative, partitions data into K clusters, scales well, but requires pre-specified K and can converge to local optima.

Both algorithms rely on distance measures and are widely used in exploratory data analysis and predictive modeling.

5 References

1. Larose, D. T., & Larose, C. D. (2014). *Discovering Knowledge in Data: An Introduction to Data Mining* (2nd ed.). Wiley.
2. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.